구정은
Emily Koo

웹 서비스를 만들어 온 프론트엔드 개발자 (React · Vue)

실무 경력 약 2년 8개월. MAU 1,400만 보배드림에서 **가상 스크롤 피드와 뒤로가기 위치 복원**을 맡아 구현했습니다.

React Next.js TypeScript AI Agent(Codex)

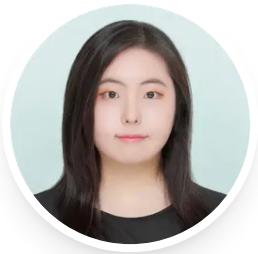
렌더링 성능
보배드림 피드에 가상 스크롤을 적용해 1,000건을 불러와도 카드 3~5개만 렌더링, 뒤로가기 위치 오차 0px

React 컴포넌트·상태 설계
디자인 시스템과 바텀시트·입력창 등 공통 컴포넌트 구축, Redux Toolkit으로 상태 설계

AI 에이전트 협업 개발
보배드림에서 구현·독립 리뷰·요구사항 대조 에이전트를 나눠 운영하고, 결과는 테스트·브라우저 측정으로 검증한 뒤 반영

웹페이지에서 보기

해당 PDF는 웹페이지에서 더 편하게 감상하실 수 있습니다.



구정은 Emily Koo

프론트엔드 개발자

2026.09 ~ 재직 중 프론트엔드 개발자

㈜보배네트워크

보배드림 전면 리뉴얼에서 가상 스크롤 피드와 뒤로가기 위치 복원, 통합 검색, 마이페이지를 개발하고 론칭까지 QA 대응

2024.07~2025.06 SSAFY 전공자 Java 트랙 수료

삼성 청년 SW·AI 아카데미

프로젝트 우수상 2회 수상, 삼성 SW 역량테스트 (A형), 2월 이달의 동료상

2024.12 SQLD 자격증 취득

한국데이터산업진흥원

데이터베이스 설계 및 관리 역량 인증, SQL 개발자 자격 취득

2021.08 ~ 2024.01 프론트엔드 개발자

㈜가온아이

Vue.js 기반 그룹웨어 PC 메신저 개발. 조직도/그룹/메신저 기능 개발, 쪽지 기능 전담, 다국어 리소스 통합, 배포와 유지보수

2021.02 ~ 2021.08 Java 개발자 양성 과정 수료

한국소프트웨어인재개발원

특모범상 수상, Spring 기반 웹 개발 역량 확보, Java 개발자 양성 과정 이수

2016.03 ~ 2021.08 경영학 학사

홍익대학교

경영학 전공

기술과 역량

실제로 사용한 프로젝트와 함께 적었습니다.

Technical Skills

React / Next.js

프로젝트

Chain G(Next.js, PWA): 디자인 시스템·공통 컴포넌트, 메인 로딩 7초 → 0.37초

Vibe Editor(React): VS Code 확장 프로그램 화면, WebView 간 상태 동기화

Redux Toolkit으로 상태 관리

TypeScript

실무·프로젝트

보배드림 리뉴얼 실무 사용

팀 프로젝트에서 API 응답 타입(ApiResponse<T>) 정의

React Native

팀 프로젝트(7주)

다와따: 실시간 위치 공유 앱 주요 화면

지도·캘린더를 WebView 공용 화면으로 전환

테스트·검증

실무

Node.js test runner로 상태 로직 테스트 작성

Playwright로 브라우저 동작·API 호출 수 측정

Vue.js / Nuxt

실무 약 2년 8개월

보배드림 리뉴얼(Nuxt 4, Vue 3): 피드 무한 스크롤, 통합 검색, 마이페이지

AI 도구

실무

AI 결과는 테스트와 브라우저 확인 후 반영(구현·리뷰 역할 분리)

가온아이(Vue, Nuxt, JavaScript):
그룹웨어 PC 메신저 기능 개발

포인터·드래그 이벤트 버그 수정 경험

Soft Skills

협업

- 비개발자 QA 리포트를 받아 처리 순서를 정하고 대응
- 백엔드와 API 사용 방식을 확인한 뒤 구현
- 영어만 쓰는 동료와 용어집·네이밍 규칙을 만들어 협업

문제 해결

- 원인이 여러 개 겹친 버그를 하나씩 분리해 수정
- 측정값으로 개선 여부 확인

어학

- 일본어 JLPT N1
- 영어 업무 소통 가능
- 6개 국어 다국어 리소스 통합 경험

프로젝트

React 팀 프로젝트와 프론트엔드 실무에서 직접 개발한 화면과 문제 해결 과정입니다.

기여도는 기획·개발·디버깅 등 주요 기여 영역 기준으로 산정했습니다.

보배드림 전면 리뉴얼

2026.09 ~ 재직 중 | 6명(개발자 4, 디자이너 1, 팀장 1)

㈜보배네트워크 사원(계약직) · 프론트엔드 개발

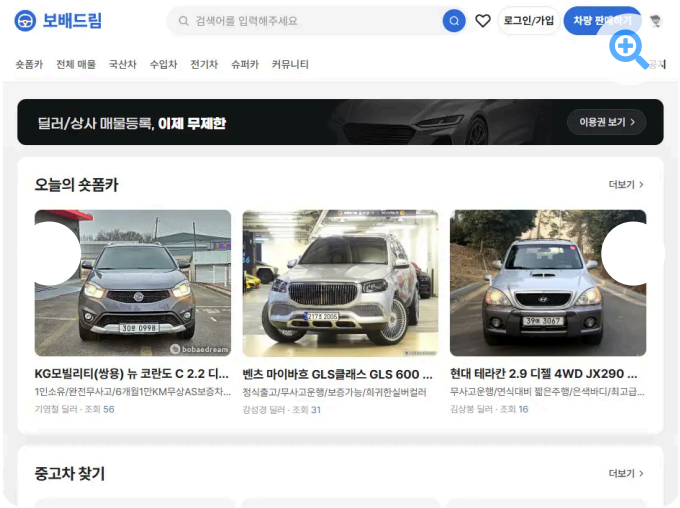
가상 스크롤 피드와 뒤로가기 위치 복원(단독 설계·개발, 기여도 100%), 통합 검색(/search) 신규 페이지, 한글 카테고리 URL 라우팅(백엔드와 협업), 마이페이지·회원 정보, 중고차 목록·상세, 론칭까지 QA 대응

MAU 1,400만 자동차 커뮤니티·중고차 서비스의 전면 리뉴얼. 기존 팀원이 만든 초기 구조 위에서 신규 화면을 개발하고, 성능은 테스트와 브라우저 측정으로 검증함.

오차 0px
1,000번째 글 뒤로가기 복원

3~5개
1,002건 로드 시 렌더링 카드

16.8ms
스크롤 프레임 간격 p95



Nuxt 4 · Vue 3 · TypeScript · TanStack Virtual · es-hangul · Playwright · Codex

트러블 슈팅

커뮤니티 피드 무한 스크롤과 뒤로가기 위치 복원 (단독 설계·개발, 기여도 100%)

⚠ **현상** 카드 높이(본문, 투표, 이미지, 영상)가 제각각이라 글 상세에서 뒤로가기를 하면 보던 위치를 정확히 복원하기 어려움

해결 - TanStack Virtual로 가상 스크롤 적용, 카드 실측 평균을 근사한 추정 높이(273.2px)로 위치 계산
- 최대 1,200건 보관, 캐시가 있으면 뒤로가기 시 목록을 다시 요청하지 않음
- 커뮤니티를 복제한 테스트 페이지와 성능 로깅 페이지(긴 작업, 입력 지연, 메모리, 프레임 간격)로 성능 저하 여부 확인
- 테스트 코드 작성(상태 로직 테스트 90개 통과)

성과 1,000번째 글에서 뒤로가기 시 같은 위치(오차 0px)로 복원, 목록 재요청 0회
- 1,002건을 불러온 상태에서도 렌더링 카드 3~5개, 스크롤 프레임 간격 중앙값 16.7ms·p95 16.8ms
- 측정: 2026-11-09 Playwright, 로컬 개발 서버, PC 1440×900

통합 검색 API 호출 91회 → 2회

⚠ **현상** 중고차·커뮤니티 결과를 함께 보여주는 신규 검색 페이지에서, 검색 1회에 API가 91회 호출됨

원인 검색 전용이 아닌 기본 차량 필터 API를 여러 번 조합해 결과를 만드는 구조였음

해결 - 백엔드 담당자에게 맞는 API와 사용법을 확인해 검색 API 2개(중고차 1, 커뮤니티 1)만 쓰도록 수정
- 모호했던 설계 문서를 "더보기 클릭으로만 추가 로드, 뒤로가기 위치 복원은 유지"처럼 행동 단위로 다시 작성

성과 검색 1회당 API 91회 → 2회, 스크롤 시 추가 호출 0회

한글 카테고리 URL 라우팅 (6종, 백엔드와 협업)

- 기존 코드는 영문 카테고리만 인식해 /car/list/슈퍼카 를 제조사 이름으로 잘못 처리함
- 국산차·수입차·올드카·슈퍼카·쇳폼카·전기차 6개 카테고리를 한글 주소로 열 수 있는 /car/list/{카테고리}/{제조사}/{모델그룹}/{모델} 구조로 백엔드와 함께 라우팅 구현, 기존 영문·제조사 주소도 유지
- 수정 전후 측정 결과 한글 주소도 영문 주소와 같은 API 4회로 동작

PC·모바일 드래그 정렬 안정화 (원인 3가지)

- 고정 항목을 앞에 두는 정렬 로직이 사용자가 바꾼 순서를 되돌림 → 사용자 편집 순서를 우선 적용
- PC에서 브라우저 기본 드래그가 포인터 이벤트를 가로챔 → 기본 드래그를 막고 이벤트를 window에서 추적
- 드래그와 무관한 포인터 입력이 드래그를 종료시킴 → 같은 pointerId일 때만 종료
- PC·모바일 모두 정상 동작, QA 리포트 해결

마이페이지 신규 개발과 API V1 → V2 전환

- 디자인 시안 확정 전에 기본 디자인으로 기능부터 개발하고, 시안이 나오면 디자인을 입힘
- 백엔드 API가 V1에서 V2로 바뀔 때 화면 코드는 고치지 않고 API 연결만 교체(전환 커밋 7건)
- 론칭 직전 하루 최대 7건 들어오는 QA 리포트의 처리 순서를 팀장에게 위임받아 "접수순 → 시안 확정 화면 → V2 전환" 기준으로 대응

Chain G

GitHub

친한 사람들과 돈 문제로 어색하지 않도록, 가볍지만 신뢰할 수 있게 정리해주는 코리빙 서비스

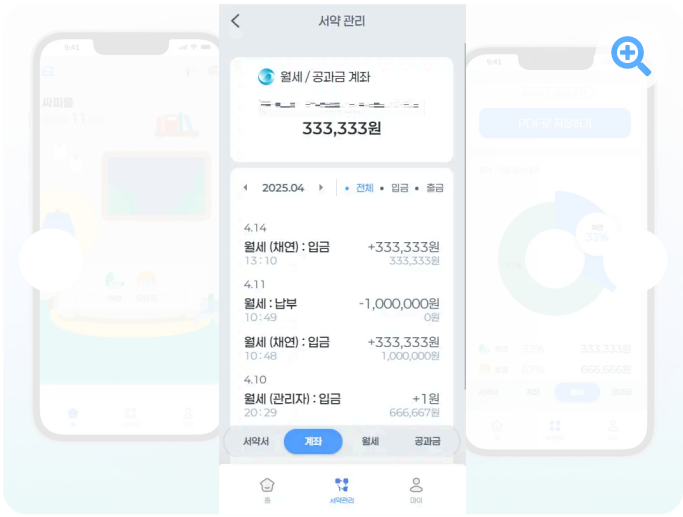
2025.02 ~ 2025.04 (7주) | 6명(기여도 20%)

프론트엔드 개발 / 디자인 시스템·공통 컴포넌트 개발

7초 → 0.37초
메인 로딩

100점
Lighthouse 접근성·SEO 등 4개 항목

FCM 연동
실시간 반영



React | Next.js | Redux Toolkit | Emotion | Design System | PWA | Service Worker | D3.js | FCM

트러블 슈팅

메인 페이지 로딩 성능 개선 (DOMContentLoaded 7초 → 0.37초)

⚠ **현상** 메인 페이지 첫 진입 시 DOMContentLoaded까지 7초가 걸리고, 새로고침에도 캐시가 반영되지 않음

🔍 **원인** 초기 리소스가 PNG, 커스텀 폰트, 공지 렌더 등으로 과도하게 무거움 (164 requests, 3.7MB transferred)

🛠 **해결**

- 이미지 리소스를 PNG → SVG로 전환하여 용량 절감
- 웹폰트는 굵기 3종만 포함해 로딩량 축소
- 공지사항 컴포넌트는 비동기 로딩으로 전환
- Service Worker 캐시 전략을 통해 재접속 시 리소스 재사용

⚡ **성과** DOMContentLoaded 기준 7초 → 0.37초로 개선 (Chrome DevTools)

📖 **회고** 최초 진입과 새로고침 시 네트워크 로그를 분리해 관리하고, 캐시 대상 우선순위를 정하는 전략이 필요

로딩 페이지의 속도 개선

⚠ **현상** 로딩 화면 자체가 무거워 첫 화면이 늦게 나타남

🔍 **원인** **Emotion 스타일 컴포넌트, 외부 폰트, SVG 애니메이션이 초기 렌더링 블로킹 요소로 작동함**

🛠 **해결**

- 로딩 페이지는 별도 구성, Emotion 제거 후 순수 HTML/CSS로 전환
- 배경 이미지 → CSS 그라데이션, 애니메이션은 inline SVG로 구성

⚡ **성과** **로딩 화면이 바로 나타나도록 개선 (메인 로딩 전체는 DOMContentLoaded 7초 → 0.37초)**

📖 **회고** 최초 페이지에는 스타일/이미지를 줄이고, HTML 기반 뼈대만 먼저 그리는 설계가 필수

PWA, Service Worker 적용해 웹앱 최적화

⚠ **현상** 앱이 매번 리소스를 새로 불러오고, 알림 기능이 제공되지 않음

🔍 **원인** 캐싱 전략과 푸시 메시지 시스템이 부재

🛠 **해결**

- Service Worker 도입으로 SVG·폰트 등 정적 리소스 캐싱
- FCM(Firebase Cloud Messaging) 연동으로 실시간 푸시 알림 구성
- manifest.webmanifest 구성으로 홈 화면 추가(PWA)

⚡ **성과** 웹앱 사용성을 모바일 네이티브에 근접하게 향상

푸시 알림, 오프라인 대응, 빠른 실행이 가능해짐

📖 **회고** PWA의 핵심은 UX 최적화가 아니라 캐시 전략과 네트워크 제어임

API 구조 일원화로 인증·에러 처리 자동화 및 타입 안전성 확보

⚠ **현상** API 호출 시마다 인증 토큰 주입과 에러 처리를 반복해야 함

🔍 **원인** axios 인스턴스 없이 각 API 파일마다 요청 설정을 따로 관리하고 있었음

🛠 **해결** - api.ts에 단일 axios 인스턴스를 생성해 모든 API 요청을 통합

- 인터셉터를 통해 accessToken 자동 주입, 에러 발생 시 공통 모듈 handleDefaultError() 호출
- getRequest<T>, postRequest<T> 등 제네릭 기반의 API 함수로 타입 안정성 확보
- 백엔드와 공통 응답 스키마(success/fail + data 구조) 협의해 ApiResponse<T>로 정형화

⚡ **성과** API 인증 흐름이 단순화되고, 에러 처리 중복 제거로 유지보수성과 개발 속도 모두 향상됨

📖 **회고** 초기 API 설계 단계에서 프론트-백 협업을 통한 구조 통일이 이후 전체 생산성을 좌우함

Vibe Editor

GitHub

2025.04 ~ 2025.05 (6주) | 6명(기여도 25%)

프론트엔드 개발 / VS Code 확장 프로그램 개발

VS Code에서 코드/디렉토리/로그 스냅샷을 찍고, AI 도움으로 기술 블로그 초안을 Notion에 게시하는 확장 프로그램



React | Next.js | TypeScript | VSCode Extension | Notion API | AI : Prompt Engineering | Jenkins | UX Writing | Figma

트러블 슈팅

Redux의 WebView 전용 구조로 인해 VSCode Extension 전역 상태 공유에 실패

⚠ **현상** Webview에서 Redux로 관리하던 전역 상태 데이터가 TreeView 등의 다른 영역에서 동기화되지 않음

🔍 **원인** **Redux는 React 기반 WebView 내부에서만 동작**하며, WebView 간 또는 TreeView 등 외부 UI 컴포넌트에서는 Redux state 접근이 불가

🛠 **해결** - ExtensionContext.globalState(VSCode Extension의 전역 저장소)를 사용해 공통 상태 데이터를 저장

- getLocalTemplates(), getLocalTemplate() 등 유틸 함수를 생성해 상태 읽기/갱신 로직 구현
- **postMessage**로 상태 변경을 통지해, 다른 WebView도 동기화
- 사용자 설정은 VSCode 설정 API를 래핑한 클래스로 전역 관리하며, settings.json에 저장

⚡ **성과** - Webview, TreeView 등 Extension 전역에서 **상태 일관성 확보**

- 모든 워크스페이스에서 캐시 데이터 공유
- 데이터 동기화코드를 별도 함수로 분리, 반복 코드 최소화
- 로그아웃 시 캐시·secret 일괄 초기화로 데이터 잔존 문제 방지

📖 **회고** 캐시 데이터의 호출 실패 시 경고 메시지에 재시도 또는 설정 확인 등의 해결책까지 명시하면 사용자 혼란을 줄이고 UX 완성도가 높아졌을 것

VSCode Extension의 쿠키 미지원으로 Oauth2 리디렉션 토큰 처리 실패

⚠ **현상** Oauth2 인증 후 AccessToken을 쿠키에 담아 전달하더라도, VSCode Extension 내부에서는 해당 쿠키에 접근할 수 없어 인증 불가

🔍 **원인** VSCode Extension은 WebView가 iframe + sandbox 환경에서 동작하며, 쿠키/세션이 차단됨

🛠 **해결** - 클라이언트인 VSCode Extension 내에서 **로컬 서버를 실행**(localhost:5013)함

- 외부 브라우저에서 로컬 서버를 열고 AccessToken을 URL 파라미터로 전달받아 직접 수신하도록 처리

- 서버는 /callback 엔드포인트로 AccessToken을 전달받아, Extension 내부의 **secret storage**에 안전하게 저장

- 로그인 성공 메시지를 띄운 뒤, 로컬 서버를 통해 VSCode Extension 내부로 상태 전달 및 initFetchData 트리거 실행

⚡ **성과** - 외부 토큰 유출 없이 Extension 내부에 AccessToken 저장 가능

- 인증 상태를 모든 WebView에 일관되게 전파하여 사용자 흐름 유지

📖 **회고** 정적 포트(5013) 대신 동적 포트를 사용했다면 포트 충돌 가능성 없이 더 안전하게 서버를 열 수 있었을 것

Tailwind의 빌드 타임 색상 정의가 VSCode 테마의 실시간 반영을 막는 이슈

⚠ **현상** VSCode의 테마를 바꿔도, Extension 내의 색상이 즉시 반영되지 않고 프로그램 재시작 후에야 적용됨

🔍 **원인** 테마 색상은 CSS 변수(var)로 제공되는데, Tailwind config의 theme 속성에 정의된 색상은 **빌드 타임에만 적용**되어 **런타임의 변수 변경**을 감지하지 못함

🛠 **해결** 색상 관련 CSS는 별도의 global.css에서 정의하고, 테마 변수(var(--vscode-foreground) 등)를 직접 참조하도록 구성. Tailwind는 레이아웃·여백·타이포 등 비색상 영역에서만 사용

⚡ **성과** 테마 변경이 즉시 반영되어, 재시작 없이 UX 흐름 유지 가능 (변경 반영 시간 0.1초 이내)

📖 **회고** 색상 클래스 정의 시 Tailwind의 명명 규칙(bg-, text-)을 활용했다면 CSS 유지보수성과 일관성을 더 확보할 수 있었을 것

Jenkins 프론트엔드 파이프라인: 캐시 잔존과 빌드 실패 코드 머지

⚠ **현상** - 이전 빌드의 캐시 파일이 남아 빌드해도 결과물이 올바르지 않음

- 빌드에 실패하는 코드가 머지된 뒤에야 자동 배포 단계에서 실패가 드러남

🛠 **해결** - 빌드 명령을 "캐시 정리 후 빌드"와 "빌드만" 두 가지로 나눔

- 머지 시점에 빌드가 되지 않으면 배포하지 않도록 파이프라인 수정

⚡ **성과 빌드되지 않는 코드가 프로덕션에 머지되는 일을 막음**

프롬프트 전환 시 입력 상태 초기화 실패

⚠ **현상** 프롬프트 전환 후에도 form 값이 이전 상태에 머무름

🔍 **원인** React Hook Form의 reset 타이밍이 form 초기값(defaultPrompt)의 변화를 감지하지 못함.

🛠 **해결** - defaultPrompt 변경 시 useMemo + useRef로 변경 감지 → reset() 호출로 폼 초기화
- preview에는 postMessage로 전환된 prompt 정보 전달하여 UI 동기화

⚡ **성과** 프롬프트 전환 시 UI 전체가 정확하게 초기화되어 사용자 혼란 제거

📖 **회고** 폼 상태 관리를 위해 FormProvider를 적용했다면 중복된 훅 구조나 reset 타이밍 제어를 더 쉽게 처리할 수 있었을 것

스냅샷 변경 사항 실시간 반영 누락

⚠ **현상** TreeView에서 snapshot을 추가하거나 수정해도, form과 preview에 즉시 반영되지 않음

🔍 **원인** 폼에 사용되는 snapshot 데이터는 defaultPrompt 내 snapshotAttachList와 별도로 관리되어 동기화가 누락됨

🛠 **해결** attachId 기준으로 snapshotList와 병합 후 form 갱신 후 동기화 유도

⚡ **성과** form과 preview가 snapshot 추가/삭제 직후에도 정확히 반영되어, 실시간 UX 유지

📖 **회고** snapshot 변경 이후 서버 반영까지 자동 처리되도록 debounce 흐름과 자동 저장 로직이 병행되었으면 더욱 완성도 높은 UX를 제공할 수 있었을 것

다와따

GitHub

2025.01 ~ 2025.02 (7주) | 6명(기여도 20%)

프론트엔드 주요 화면 개발 / UI 디자인

실시간 위치 공유 및 최적 장소 추천을 통해 약속 시간을 효율적으로 조율하는 서비스

240초 → 0.5초
추천 결과 첫 표시

38분 → 2분
APK 빌드



React Native | Redux | WebSocket | S3 | Gradle

트러블 슈팅

캐시된 추천 결과를 먼저 보여 줘 첫 표시까지 240초 → 0.5초 (API 자체 응답 시간은 그대로)

- 장소 추천 API(팀원 담당)는 여러 명의 위치를 바탕으로 최적 경로를 계산해 응답에 4분이 걸림. 계산 특성상 API 자체를 줄이기 어려움
- 많은 변수를 한 번에 넣던 호출을, 변수를 나눠 부분적으로 입력하고 나누어 처리하는 API 호출 방식으로 교체
- 사용자가 기다리지 않도록 먼저 약속 기본 정보를 입력하도록 개선
- Redux 캐싱을 활용해 기존 추천 장소를 즉시 노출, 이후 백그라운드에서 최신 데이터 갱신
- Redux Persist Storage를 적용하여 재접속 시에도 추천 내역 유지, 중복 API 요청 방지

지도·캘린더를 WebView 공용 화면으로 전환

- Android·iOS 네이티브 컴포넌트로 따로 구현하니 플랫폼마다 화면이 달라지고 분기 코드가 늘어 유지보수가 어려움
- 지도·캘린더 페이지를 WebView 기반 공용 컴포넌트로 옮기고 반응형으로 만들어 PC·태블릿까지 대응
- 플랫폼별 분기를 줄이고 Android·iOS에서 같은 화면 제공

Amazon S3 Presigned URL을 활용한 이미지 업로드 개선

- Presigned URL을 사용한 프로필 이미지 업로드 과정에서 바이너리 데이터 인증 오류 발생
- 업로드 전 이미지 리사이징 및 PNG 변환 적용 → 서버가 요구하는 확장자 규격을 준수하도록 수정

CI/CD 자동화 및 APK 빌드 속도 개선

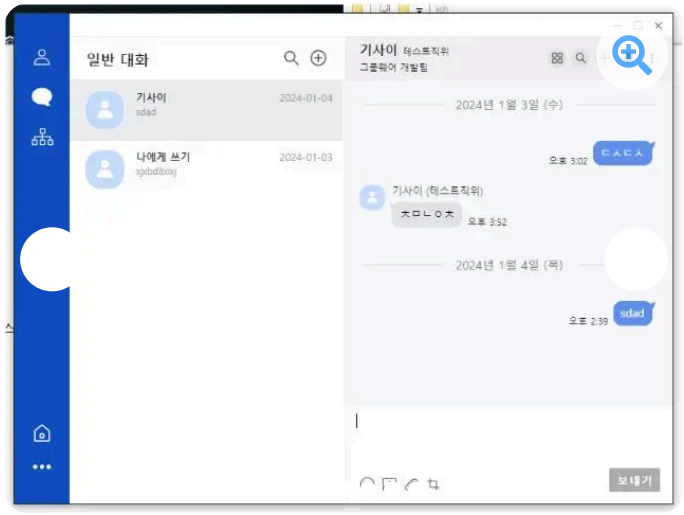
- Android Native Module, FCM, Kakao SDK 추가 후 빌드 시간이 급격히 증가(38분 소요)
- Gradle 빌드 설정 조정 → APK 빌드 시간 38분 → 2분

bizmekaTalk 3.0

2021.11 ~ 2024.01 | ㈜가온아이 MI개발부 중 3~6명 (기여도 30%)

개발: 조직도/그룹/메신저 기능 개발, 쪽지 기능 전담 개발
유지 보수: 오류 수정, QA, 배포(Pascal로 설치 파일 제작)
협업: 영어만 쓰는 동료 합류 후 팀원들과 한·영 개발 용어집과 네이밍 규칙을 함께 정리하고, 업무별 담당자를 정하자고 제안해 리뷰·조율 혼선을 줄임

사용자 30만 명 규모 그룹웨어(비즈메카)의 신규 PC 메신저 클라이언트 프론트엔드 개발과 운영을 담당. 쪽지 기능을 전담 개발하고, 설계 및 기술 스택 선정 단계부터 참여함. 고객사 요구에 따른 옵션 기능 구현과 SaaS 대응을 위해 SSR SPA 구조로 개발함.



Vue.js JavaScript Nuxt Vuetify i18n C# WPF Webview2

트러블 슈팅

창 접기·펼치기 시 UI 상태 복원 적용

- KumTalk에서 만든 UI 상태 복원 방식(스토어 전체를 Session Storage에 저장)을 bizmekaTalk 3.0에 적용
- 창을 반으로 접고 펼쳐도 열린 대화방, 스크롤 위치, 선택값, 입력 중 내용, 레이아웃이 그대로 유지됨

일부 환경에서 절전모드 미인식

Windows OS별 절전모드 구현 차이를 고려해 와이파이 변경 감지 기능을 수정함

Webview의 새 창끼리의 데이터 동기화 문제

Session Storage를 활용하여 Webview 간 데이터 동기화 기능 구현

Pratama

2023.06 ~ 2023.09 | ㈜가온아이 MI개발부 중 4명 + 외부 번역 인력 (기여도 60%)

다국어 처리, 리소스 파일 설계 및 적용, UI/UX 커스터마이징, QA 및 유지보수

그룹웨어 솔루션의 글로벌 출시를 위한 다국어 지원 및 커스터마이징 프로젝트. PC와 모바일 리소스를 통합하고, 반응형 UI로 개편. 사용자 기본 언어 기반 설치 파일 제작 등 6개 국어 리소스를 통합함.



Vue.js JavaScript Nuxt i18n Vuetify C# WPF Webview2

트러블 슈팅

날짜 표기 방식 불일치

인도네시아 언어의 특성을 고려하여 날짜 형식을 재조정함

UI 대비 과도한 번역 문자열

UI를 반응형으로 수정하고, 번역문 조정 작업을 외부 번역 인력과 조율하여 해결함

KumTalk

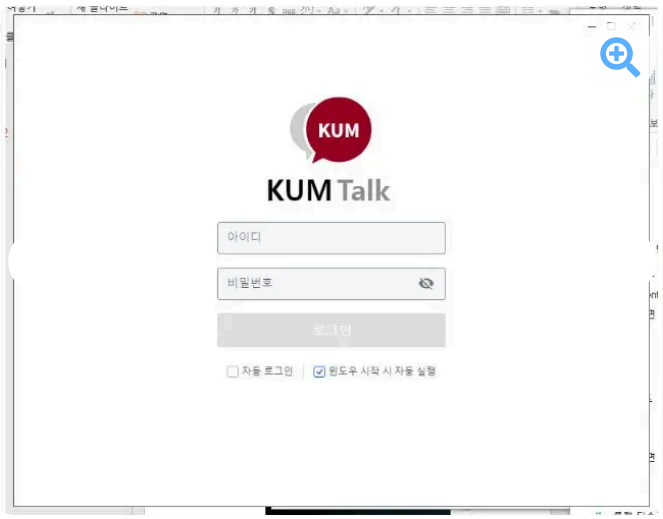
2021.11 ~ 2022.09 | ㈜가온아이 MI개발부

조직도/그룹/설정 기능 개발, 대화방 및 환자 대화방 오류 수정, 유지보수 및 배포

고려대학교의료원의 그룹웨어 클라이언트 커스터마이징 프로젝트. 조직도와 그룹, 설정 기능 개발을 비롯하여, 환자 대화방 기능을 Vue 컴포넌트로 모듈화. 자사 그룹웨어 연동 기능과 설치 파일 제작, 원격 지원 및 운영까지 담당.

6명 (메인 2명, 서브 4명)
팀 규모

25%
기여도



Vue.js JavaScript Nuxt Vuetify C# WPF Webview2

트러블 슈팅

창 접기·펼치기 시 UI 상태 전체 복원

- 메신저 창을 반으로 접고 펼 때마다 WPF 창 크기가 바뀌며 Webview가 다시 그려져, 화면 상태가 모두 초기화됨
- 화면 상태(열린 대화방, 스크롤 위치, 선택값, 입력 중 내용, 레이아웃)를 스토어에 모으고, 스토어 전체를 Session Storage에 저장해 접기 전 상태 그대로 복원
- 간호사 여러 명이 한 PC에서 번갈아 로그아웃·로그인하는 환경이라 Local Storage를 쓸 수 없어, 계정마다 상태가 섞이지 않도록 Session Storage 사용
- 이후 같은 방식을 bizmekaTalk 3.0에 적용

메시지 내 링크 검색 시 UI 깨짐 현상

DOM 구조 분석 후 CSS 리팩토링을 통해 해결

Props 남용으로 인한 중복 코드

Props를 여러 단계로 넘기던 구조를 이벤트 버스(Vue 2 당시 방식)로 정리해 중복 제거

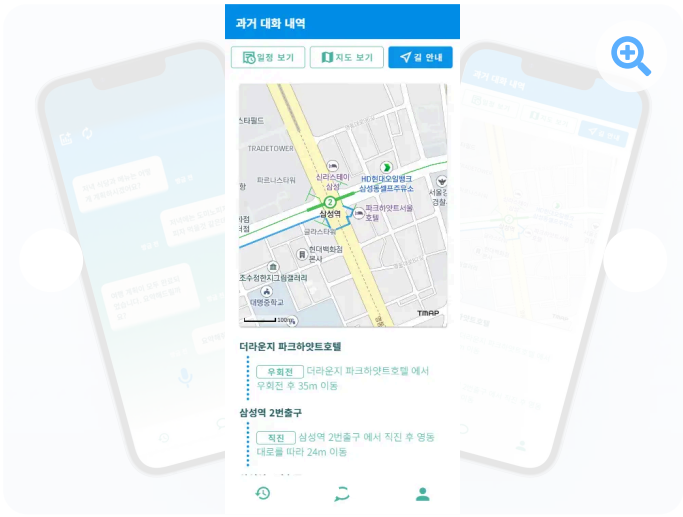
트매

GitHub

AI LLM을 활용한 음성 대화 트립 매니징 서비스

2024.10 ~ 2024.11 (2주) | 2명(기여도 55%)

기획 / UI 개발 / 음성 대화 기능 개발



Vue3 | Spring Boot | OpenAI API | Prompt Engineering | WebSocket | TMap

트러블 슈팅

AI 리포트를 마크다운 렌더링으로 전환해 필요한 컴포넌트 약 100개 → 15개 안팎

- 생성형 AI가 HTML 리포트를 직접 만들게 하니 리포트마다 스타일이 달라, 모양을 맞추려면 컴포넌트가 백여 개 필요함
- AI 응답을 마크다운으로 받아 HTML로 다시 렌더링하고, 마크다운의 한정된 요소에 맞춘 디자인 시스템 컴포넌트를 만듦
- 리포트 모양이 균일해지고 필요한 컴포넌트가 15개 안팎으로 줄어듦

STT 기능을 AI를 사용하고 RestAPI를 사용할 경우 딜레이 과다 발생

- STT를 브라우저 내장 기능으로 사용하되, 무음 인식 교정 컴포넌트 추가 구현
- RestAPI 대신 WebSocket을 이용해 속도 최적화

WebSocket으로 소리와 JSON을 동시에 보낼 경우 Spring 핸들러 미인식 오류

- STT : FE단에서 처리하고 TextMessage만 BE로 전송
- TTS : BinaryMessage를 AudioBlob 형식으로 전송함과 동시에 해당 Json을 TextMessage로 전송한 뒤 싱크 맞추기 작업 추가

프롬프트가 길 경우 Open AI 타임 아웃 이슈 발생

- 프롬프트 엔지니어링을 이용한 토큰 최적화
- 실제로 다루는 메시지의 가공을 통한 토큰 수 최적화

Contact

Email: jemilykoo@gmail.com

GitHub: JEmilyKoo

Portfolio: https://portfolio-beta-seven-87.vercel.app/v1

Blog: velog.io/@jemilykoo